Chapter 20: Statistical Learning Methods

1. What is Statistical Learning?

- Statistical learning uses data to automatically build models that make predictions.
- Learns a function h from examples rather than being explicitly programmed.
- Foundations: probability theory, statistics, and optimization.
- Used when explicit rules are hard to write.

Key Idea: A learning algorithm uses past data to find a good hypothesis (model) that generalizes to unseen examples.

2. Supervised Learning

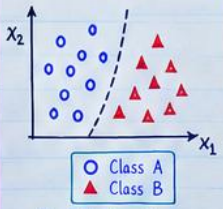
- Given a set of labeled examples $\{(x_1, y_1), \ldots, (x_n, y_n)\}$.
- Each example has input features x and a target output y.
- Learn a function (hypothesis) h such that $h(x) \approx y$.
- Used for both classification (discrete targets) and regression (continuous targets).

Goal: find h in a hypothesis space $\mathcal{H}$ that minimizes the expected loss on new, unseen examples.

3. Classification vs. Regression

Classification

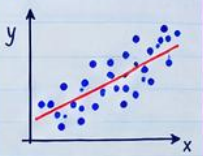
- Target y is a discrete class label.
- Learn $h(x)$ that predicts a class.
- Example: email spam detection (spam / not spam).



Classification: learn a decision boundary.

Regression

- Target y is a continuous numeric value.
- Learn $h(x)$ that predicts a real number.
- Example: house price prediction.



Regression: learn a continuous function.

4. Examples of Learning from Labeled Data

Application	Input Features (x)	Target (y)	Type
Email spam detection	Word counts, metadata	Spam / Not spam	Classification
Medical diagnosis	Patient measurements	Disease / No disease	Classification
Handwritten digit recognition	Pixel values	Digit (0–9)	Classification
House price prediction	Size, location, rooms	Price (e.g. $)	Regression
Temperature prediction	Past weather data	Next day temperature	Regression
Stock price prediction	Historical prices	Next price	Regression

5. Features and Target Variable

- Each example is a feature vector $x = (x_1, x_2, \ldots, x_d)$
- Features are measurable properties of the input (e.g., numeric, categorical, binary).
- Target variable y is the value we want to predict.
- The learned function (hypothesis) is written as $h(x)$.
- For classification: $h(x)$ outputs a class label or a probability $P(y \mid x)$.
- For regression: $h(x)$ outputs a real-valued prediction.

Notation: training examples $D = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ from an unknown distribution $P(x, y)$.

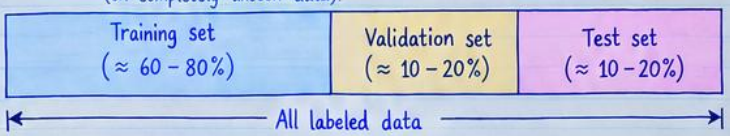
6. Hypothesis Space

- A hypothesis h is a candidate function mapping inputs to outputs: $h : X \to y$ (written $h(x)$).
- The hypothesis space $\mathcal{H}$ is the set of all possible functions the algorithm can choose from.
- The choice of $\mathcal{H}$ determines the flexibility of the model (e.g., linear functions, decision trees, neural networks).
- A more expressive $\mathcal{H}$ can fit more complex patterns, but may also overfit.

Example: linear regression hypothesis: $h(x) = w^T x + b$
Parameters w, b are learned from data.

7. Training, Validation, and Test Sets

- **Training set:** used to learn the model (fit parameters).
- **Validation set:** used to select models / tune hyperparameters (e.g., regularization strength, model complexity).
- **Test set:** used to estimate final generalization performance (on completely unseen data).



8. Generalization

- A good model should perform well on new, unseen examples drawn from the same distribution.
- Generalization error is the expected loss on new data:

$$E_{x,y}\left[L(h(x), y)\right]$$

- Since the true distribution is unknown, we estimate generalization using a held-out test set.

Goal: minimize generalization error, not just training error.

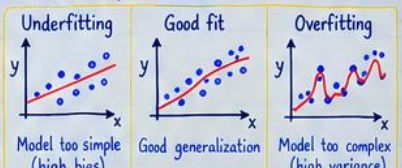
9. Noise in the Data

- Real-world data often contains noise due to measurement errors, missing information, or inherent randomness.
- Noise can make it impossible to learn a perfect model (even with infinite data).
- A good model should be robust to noise and capture the underlying pattern rather than the noise.

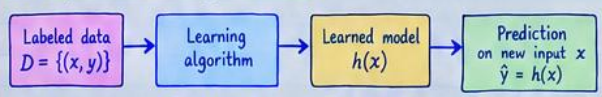
Result: even the best possible model has non-zero error when data is noisy.

10. Underfitting vs. Overfitting

- **Underfitting:** model is too simple to capture the true pattern (high bias).
- **Overfitting:** model is too complex and fits noise in the training data (high variance).



11. Supervised Learning Workflow



- Collect and preprocess data
- Extract features
- Choose hypothesis space $\mathcal{H}$
- Minimize training loss
- Tune hyperparameters using validation set
- Final hypothesis after training
- Can output class label or numeric value
- Class label (classification) or real value (regression)

12. Loss Function (Examples)

- Measures how bad a prediction is.
- Training tries to minimize the average loss on the training set.

Task	Common Loss $L(y, h(x))$
Classification (binary)	0-1 loss: $L(y, h(x)) = \begin{cases} 0 & \text{if } h(x) = y \\ 1 & \text{otherwise} \end{cases}$
Classification (probabilistic)	Log loss (cross-entropy): $L(y, h(x)) = -\sum_k y_k \log h_k(x)$
Regression	Squared error: $L(y, h(x)) = (y - h(x))^2$

13. Key Takeaways

1. Statistical learning learns from labeled data.
2. Used for classification and regression.
3. Choose features, hypothesis space, and loss function.
4. Split data into training, validation, and test sets.
5. Aim for good generalization.
6. Real-world data is noisy.
7. Avoid underfitting and overfitting.
8. The right model balances bias and variance.

Chapter 20 : Statistical Learning Methods

1. What is Statistical Learning?

- Learn a function from data (examples).
- Given training examples (x, y), find a hypothesis $h(x)$ that generalizes to new, unseen examples.
- Uses probabilistic and statistical assumptions.
- Two main problems: regression (predict real value) and classification (predict class).

Key Idea: Choose hypothesis parameters that fit the training data well and generalize to new data.

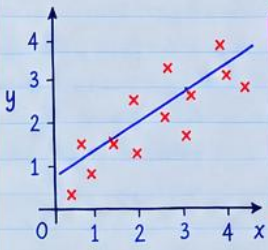
2. Linear Regression Intuition

- Predict a continuous output y from input features x.
- Assume a linear relationship:

$$y = w^T x + b$$ (single feature: $y = wx + b$)

- w are weights (parameters), b is bias.
- Finds the line (or hyperplane) that best fits the data in a least-squares sense.

3. Fitting a Line (Example)



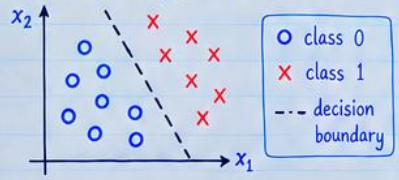
Find w, b to minimize the difference between predicted values and actual values (least squares).

4. Linear Classification

- Predict a discrete class label (e.g., 0 or 1).
- Use a linear decision boundary:

$$h(x) = \text{sign}(w^T x + b)$$

- If $w^T x + b \geq 0 \rightarrow$ class 1
  else $\rightarrow$ class 0
- Decision boundary (in 2D) is a line:



5. Logistic Idea for Class Probability

- Instead of a hard classification, predict the probability of class 1 using the logistic (sigmoid) function.

$$P(y = 1 \mid x) = \sigma(w^T x + b)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Outputs a value between 0 and 1.
- Classify as 1 if $P(y = 1 \mid x) \geq 0.5$.
- Equivalent to linear model in log-odds:

$$\log \frac{P(y = 1 \mid x)}{P(y = 0 \mid x)} = w^T x + b$$

6. Loss / Error

- Measures how well the model fits data.
- Regression: squared error

$$L(y, \hat{y}) = (y - \hat{y})^2$$

- Classification: cross-entropy (log loss)

$$L(y, p) = -[y \log p + (1-y) \log (1-p)]$$

where $p = P(y = 1 \mid x)$.

- Lower loss means better predictions.
- We usually minimize the average loss over the training set.

7. Cost Function

- Sum (or average) of losses over all training examples.
- Linear regression (least squares):

$$J(w, b) = \frac{1}{m} \sum_{i=1}^{m} (y_i - (w^T x_i + b))^2$$

- Logistic regression (cross-entropy):

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^{m} [y_i \log p_i + (1-y_i) \log (1-p_i)]$$

- Goal: find parameters (w, b) that minimize J.

8. Parameter Estimation

- Find parameters w, b that minimize the cost function $J(w, b)$.
- Often done using optimization algorithms such as gradient descent.
- The result is the best-fitting model on the training data.

Notation:
- m : number of training examples
- x_i : input feature vector
- y_i : target value (real or class)
- w : weight vector
- b : bias (intercept)

9. Gradient Descent Idea

- Iteratively move parameters in the direction of negative gradient (steepest decrease).

$$w \leftarrow w - \alpha \nabla_w J(w, b)$$

$$b \leftarrow b - \alpha \nabla_b J(w, b)$$

- α is the learning rate (step size).
- Repeat until convergence (cost stops decreasing).

Intuition: Move parameters a little bit in the direction that reduces the cost most quickly.

10. Regularization

- Helps prevent overfitting by penalizing large weights.
- Add a regularization term to the cost:

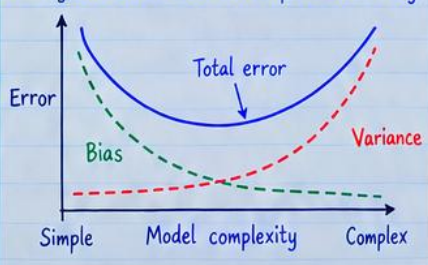
$$J_{reg}(w, b) = J(w, b) + \lambda \|w\|^2$$

(L2 regularization)

- $\lambda \geq 0$ controls the strength of the penalty.
- Encourages simpler models (smaller weights), which generalize better.
- Other forms: L1 regularization ($\lambda \|w\|_1$).

11. Bias-Variance Intuition

- Model generalization depends on a trade-off between bias and variance.
- High bias: model too simple $\rightarrow$ underfitting.
- High variance: model too complex $\rightarrow$ overfitting.



12. Summary / Key Takeaways

1. Statistical learning learns a function from data to predict values or classes.
2. Linear regression fits a line using least squares.
3. Linear classification uses a linear decision boundary.
4. Logistic regression models class probabilities with the sigmoid function.
5. Loss and cost functions measure error.
6. Parameters are estimated by minimizing the cost (e.g., gradient descent).
7. Regularization helps prevent overfitting.
8. Bias-variance trade-off affects generalization.
9. Choose an appropriate model complexity for good performance on new data.

"Good generalization means fitting the training data well and also performing well on unseen examples."

— AIMA (Ch. 20)

Chapter 20 : Statistical Learning Methods

1. Nearest-Neighbor Learning

- Instance-based (lazy) learning algorithm.
- No explicit training phase (just store examples).
- To classify a new instance, find the closest training example(s) and use their labels.
- Can also do regression (predict numeric value).

Key Idea: Similar instances are likely to have similar labels (local smoothness).

Example: Classify a new point using the label(s) of its nearest neighbors.

2. Distance Metrics

- Measure of similarity between instances.
- Common metric: Euclidean distance.

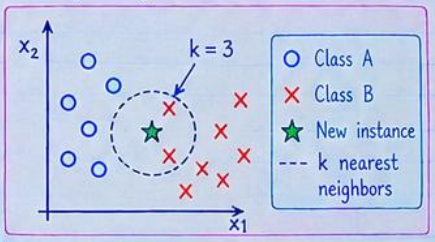
Euclidean distance (d-dimensional):

$$d(x, z) = \sqrt{\sum_{i=1}^{d} (x_i - z_i)^2}$$

- Other metrics: Manhattan (L1), Minkowski (L_p), cosine distance.
- For categorical attributes, use a suitable distance (e.g., Hamming).
- Always normalize/scale features to avoid dominance by large ranges.

3. k-Nearest Neighbors (k-NN) Intuition

- Consider the k closest training examples.
- Classification: use majority vote of k neighbors.
- Regression: use average (or weighted average).
- Choice of k controls the bias-variance tradeoff:
  - Small k: low bias, high variance (can overfit).
  - Large k: high bias, low variance (smoother).



4. Nonparametric Learning

- k-NN is a nonparametric method.
- Does not assume a fixed functional form or a fixed number of parameters.
- Model complexity grows with the amount of data (more examples → more detail).
- Flexible and can model complex decision boundaries.

Advantages:

- Simple and intuitive.
- Works for classification and regression.
- Can model highly non-linear functions.

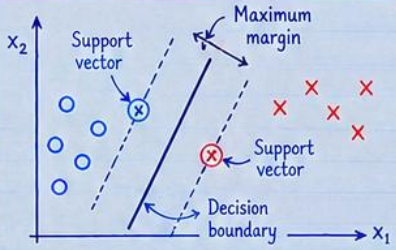
Disadvantages:

- Requires storing all training data.
- Prediction can be slow for large datasets.
- Performance depends on distance metric and choice of k.

5. Support Vector Machines (SVMs)

- Supervised learning algorithm for classification (also regression).
- Finds a linear decision boundary with the largest possible margin between classes.
- Only a subset of training examples (support vectors) determine the boundary.

Maximum Margin Idea: Choose the linear separating hyperplane that maximizes the margin to the closest training examples (support vectors).



6. Kernel Intuition

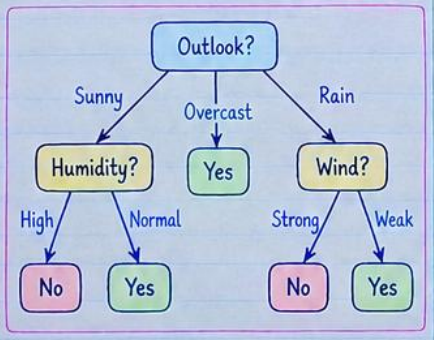
- Many problems are not linearly separable in the original feature space.
- Use a kernel function to implicitly map data to a higher-dimensional feature space where a linear boundary can separate them.
- Common kernel: Radial Basis Function (RBF)

$$K(x, z) = \exp\left(-\gamma \|x - z\|^2\right)$$

- Other kernels: polynomial, sigmoid, etc.
- Kernel trick: computes dot products in the high-dimensional space without explicitly computing the mapping $\phi(x)$.
- Results in a non-linear decision boundary in the original space.

7. Decision Trees

- Supervised learning method for classification (and regression).
- Represent a decision process as a tree.
- Each internal node tests an attribute, each branch is an outcome, and each leaf gives a class label (or numeric value).
- Easy to interpret and fast at prediction time.



8. Splitting Attributes

- Choose the attribute that best separates the training examples (most "pure" child nodes).
- Common measures:

a) Information Gain (for classification)

- Entropy: $H(S) = -\sum_i p_i \log_2 p_i$
- Information gain:

$$IG(S, A) = H(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} H(S_v)$$

- Choose attribute with highest IG.

b) Gini Index (alternative)

$$Gini(S) = 1 - \sum_i p_i^2$$

Choose attribute with lowest Gini after split.

c) For numeric attributes:

- Try possible threshold values (e.g., $x \leq t$).
- Choose split with highest information gain (or lowest Gini).

9. Comparison of Methods

Method	Model Type	Key Idea	Pros	Cons
k-NN	Nonparametric	Nearby examples	Simple, flexible	Slow prediction, needs memory
SVM	(Kernel) max-margin	Maximum margin hyperplane	Good generalization, works well in high-dim	Choice of kernel/params, less interpretable
Decision Trees	Parametric (tree model)	Recursive partitioning	Interpretable, fast prediction	Can overfit, unstable (Small data changes)

10. Key Takeaways

1. k-NN is a simple, nonparametric, instance-based method.
2. Distance metric and k are crucial for performance.
3. SVM finds a maximum margin boundary; kernels allow non-linear decision boundaries.
4. Decision trees split on attributes using measures like information gain or Gini index.
5. Each method has tradeoffs; the best choice depends on the problem and data.

"Statistical learning finds patterns in data, helping us make good predictions about new cases."

— AIMA (Ch. 20)

Chapter 20 : Statistical Learning Methods

1. Model Selection and Evaluation

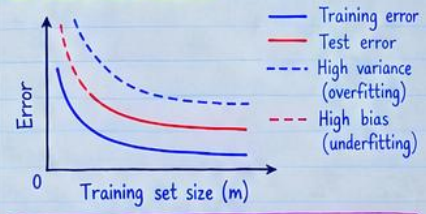
- Choose a model (hypothesis class) and its parameters using data.
- Evaluate generalization performance on unseen data.
- Need to balance underfitting and overfitting.

Key Idea: A good model generalizes well to new examples, not just the training data.

2. Training vs. Test Error

- Training error: error on the training set. Often decreases as model gets more complex.
- Test error: error on unseen data. Measures generalization performance.
- Training error usually underestimates true error; test error is an unbiased estimate when test set is independent.

3. Learning Curves



As the training set size increases, the training and test error converge. The gap indicates overfitting.

4. Cross-Validation

- Split data into k folds.
- Train on $k-1$ folds and test on the remaining fold.
- Repeat for all folds and average the test error.
- Common choice: $k = 5$ or $k = 10$.

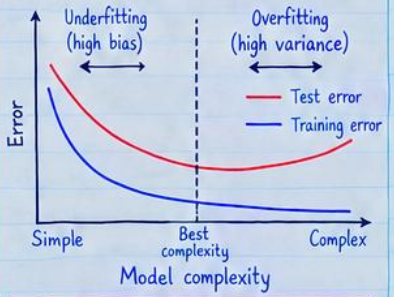
k-Fold Cross-Validation (Example: $k = 5$)

Iteration	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
1	Test	Train	Train	Train	Train
2	Train	Test	Train	Train	Train
3	Train	Train	Test	Train	Train
4	Train	Train	Train	Test	Train
5	Train	Train	Train	Train	Test

= Test set = Training set

5. Overfitting and Model Complexity

- Overfitting: model fits noise in training data. Low training error, high test error.
- Underfitting: model too simple to capture pattern. High training and test error.



Choose model complexity that minimizes the estimated test error (e.g., via cross-validation).

6. Ensemble Methods

- Combine predictions from multiple models.
- Often more accurate and robust than a single model.
- Two popular ensemble methods:
  - Bagging (bootstrap aggregating)
  - Boosting (sequentially improve models)

7. Bagging (Bootstrap Aggregating)

- Generate many training sets by sampling with replacement (bootstrap samples).
- Train a base learner on each sample.
- Combine predictions (average for regression, majority vote for classification).
- Reduces variance and helps prevent overfitting.

Example: Random Forest = bagging of decision trees + random feature selection.

8. Boosting

- Train learners sequentially, each focusing on examples that previous learners got wrong.
- Combine weak learners into a strong learner.
- Popular algorithm: AdaBoost.

AdaBoost (high-level)

1. Initialize equal weights on all training examples.
2. Train a weak learner (e.g., decision stump).
3. Increase weights on misclassified examples.
4. Repeat for T rounds.
5. Final prediction: weighted majority vote.

9. Confidence and Robustness

- Confidence: use statistical methods (e.g., confidence intervals, hypothesis tests) to quantify uncertainty in estimates.
- Robustness: performance should not change dramatically for small changes in the data.
- Ensemble methods (bagging, boosting) often improve robustness.

Bootstrap for Confidence Intervals

1. Generate many bootstrap samples.
2. Compute the statistic (e.g., mean, error) for each.
3. Use the distribution to form a confidence interval.

Also useful for estimating variance and robustness of model performance.

10. Confusion Matrix and Evaluation Metrics

Confusion Matrix (Binary Classification)

	Predicted Positive	Predicted Negative
Actual Positive (True label = 1)	True Positive (TP)	False Negative (FN)
Actual Negative (True label = 0)	False Positive (FP)	True Negative (TN)

Common Metrics

$$\text{Accuracy} = \frac{TP + TN}{(TP + TN + FP + FN)}$$

$$\text{Precision} = \frac{TP}{(TP + FP)}$$ Fraction of predicted positives that are correct.

$$\text{Recall (Sensitivity)} = \frac{TP}{(TP + FN)}$$ Fraction of actual positives detected.

$$\text{F1 score} = 2 \times \frac{(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$$

11. Mini Example: Comparing Models

Model	Training Error	Test Error (5-fold CV)	Comments
Linear Regression	0.18	0.20	Underfitting (too simple)
Decision Tree (depth = 3)	0.10	0.12	Good balance
Decision Tree (depth = 15)	0.01	0.18	Overfitting
Random Forest (bagging)	0.06	0.10	Lower variance
AdaBoost (decision stumps)	0.08	0.09	Strong performance

12. Key Formulas and Tips

- Cross-validation error:

$$E_{CV} = \frac{1}{k} \sum_{i=1}^{k} E_i$$

- Accuracy: $\frac{TP + TN}{(TP + TN + FP + FN)}$
- Precision: $\frac{TP}{(TP + FP)}$
- Recall: $\frac{TP}{(TP + FN)}$
- F1 score: $2 \times \frac{(P \times R)}{(P + R)}$

Tips:

- Use cross-validation for model selection.
- Use a separate test set for final evaluation.
- Control overfitting with simpler models, more data, or ensemble methods.
- Report multiple metrics (e.g., precision, recall, F1).
- Consider confidence intervals for robustness.

"The goal of learning is to generalize well to new, unseen examples." — AIMA (Ch. 20)

Chapter 20 : Statistical Learning Methods

1. Core Algorithms (Recap)

Main learning methods covered in this chapter:

- Linear regression (predicting continuous values)
- Logistic regression (classification)
- Naïve Bayes (probabilistic classification)
- Decision trees (classification and regression)
- Neural networks (non-linear models)
- Support Vector Machines (SVM)
- Ensemble methods (e.g., boosting, bagging)
- Reinforcement learning (briefly, as statistical learning in sequential settings)

2. Strengths and Weaknesses

Method	Strengths	Weaknesses
Linear Regression	Simple, fast, interpretable	Assumes linearity, sensitive to outliers
Logistic Regression	Probabilistic output, interpretable	Linear decision boundary
Naïve Bayes	Very fast, works well with high-dim.	Strong independence assumption
Decision Trees	Interpretable, handles non-linear	Prone to overfitting
Neural Networks	Highly expressive, learns complex fns.	Needs lots of data, hard to interpret
SVM	Effective in high dimensions	Choice of kernel, less interpretable
Ensemble Methods	High accuracy, robust	More complex, less interpretable

3. When to Use Each Method?

Method	Suitable Scenarios
Linear Regression	Predicting a continuous value with roughly linear relationship.
Logistic Regression	Binary or multi-class classification (linear boundary).
Naïve Bayes	Text classification, high-dimensional sparse data.
Decision Trees	Interpretable model, mixed feature types, non-linear data.
Neural Networks	Complex non-linear problems, large datasets (images, speech).
SVM	High-dimensional data, clear margin between classes.
Ensemble Methods	When high predictive accuracy is required.
Reinforcement Learning	Sequential decision problems with delayed rewards.

4. Important Terminology

- **Training set:** Data used to learn parameters.
- **Test set:** Unseen data to evaluate performance.
- **Features (attributes):** Input variables.
- **Target (labels):** Output variable to predict.
- **Hypothesis:** Model learned from data.
- **Parameter learning:** Finding best model parameters (e.g., weights, tree structure).
- **Loss function:** Measures prediction error (e.g., squared loss, cross-entropy).
- **Regularization:** Penalize model complexity to prevent overfitting (e.g., L1, L2).
- **Overfitting:** Fits training data too closely, poor generalization.
- **Underfitting:** Model too simple, high bias.
- **Bias–Variance tradeoff:** Tradeoff between model complexity and generalization error.
- **Cross-validation:** Estimate performance using multiple train/test splits.

5. Common Pitfalls

- **Overfitting:** model too complex, memorizes training data.
- **Underfitting:** model too simple, misses important patterns.
- **Data leakage:** using test information during training.
- **Poor feature representation:** irrelevant or missing features.
- **Ignoring regularization:** leads to overfitting.
- **Incorrect evaluation:** using training accuracy instead of test/cross-validation.
- **Imbalanced datasets:** biased predictions towards majority class.
- **Assuming independence** (e.g., Naïve Bayes) when it does not hold.
- **Not tuning hyperparameters** (e.g., C in SVM, depth in trees, learning rate in neural networks).

6. Summary of Methods

Method	Type	Key Idea	Common Uses
Linear Regression	Supervised (Regression)	Linear model $\hat{y} = w^T x$	Predict continuous values
Logistic Regression	Supervised (Classification)	Sigmoid function, probabilities	Binary/multi-class classification
Naïve Bayes	Supervised (Classification)	Bayes' rule with independence	Text, high-dim. classification.
Decision Trees	Supervised (Classification/ Regression)	Split on features to reduce impurity	Interpretable models
Neural Networks	Supervised (Classification/ Regression)	Layers of non-linear units	Complex patterns (images, speech)
SVM	Supervised (Classification/ Regression)	Maximum margin hyperplane	High-dimensional data
Ensemble Methods	Supervised	Combine multiple models (bagging, boosting)	Higher accuracy and robustness
Reinforcement Learning	Sequential Decision Making	Learn from rewards	Control, robotics, game playing

7. Likely Exam Questions

1. Compare and contrast linear regression and logistic regression. When would you use each?
2. Explain the Naïve Bayes assumption. When is it likely to work well or poorly?
3. What is overfitting? How can it be prevented?
4. Describe the bias–variance tradeoff with examples.
5. How do decision trees choose which feature to split on? (Information gain / Gini index)
6. Explain the main idea behind SVM. What is the role of the kernel?
7. How do ensemble methods (bagging and boosting) work?
8. Discuss the strengths and weaknesses of neural networks.
9. How do you evaluate a learning algorithm? What is cross-validation and why is it important?
10. Given a dataset description, which learning method would you choose and why?

8. Key Takeaways

- There is no single best learning algorithm.
- Choice of method depends on the data, task, and requirements (accuracy, interpretability, scalability).
- Simple models (e.g., linear regression, Naïve Bayes) often work well and are easy to interpret.
- Model complexity must be controlled to avoid overfitting (regularization, validation).
- Neural networks are powerful for complex, high-dimensional problems but need large datasets.
- Ensemble methods usually provide better accuracy than single models.
- Good feature representation and proper evaluation are crucial for success.
- Reinforcement learning extends statistical learning to sequential decision problems with rewards.

Generalization Error

Generalization error = $\text{Bias}^2 + \text{Variance} + \text{Irreducible error}$

- More complex models → lower bias but higher variance.
- Regularization and more data help reduce generalization error.

Typical Evaluation Metrics

- Regression: Mean Squared Error (MSE), Mean Absolute Error (MAE).
- Classification: Accuracy, Precision, Recall, F1 score, ROC curve.
- Model selection: Cross-validation, hold-out test set.
- Compare models on unseen data, not just training performance.

> "Statistical learning methods allow us to learn from data, turning experience into knowledge, and predictions, helping us make intelligent decisions in the real world."
>
> — AIMA (Ch. 20)